Lab Experiment 21 to 30

Name: PAVITHRA K **Reg no:192411027**

Course code : CSA4704 **Course name : Deep learning**

EXPERIMENT 21: FACTOR ANALYSIS AND LATENT FACTOR INTERPRETATION

AIM

To identify hidden factors influencing employee satisfaction using Factor Analysis with varimax rotation and compare the results with PCA.

ALGORITHM

1. Import the required Python libraries.
2. Generate survey data for 300 employees with 10 questions.
3. Calculate the correlation matrix.
4. Perform Factor Analysis with 2, 3, and 4 factors.
5. Apply varimax rotation and plot the factor loadings.
6. Interpret the factors based on the highest loadings.
7. Compare Factor Analysis and PCA.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.decomposition import FactorAnalysis, PCA

from sklearn.preprocessing import StandardScaler

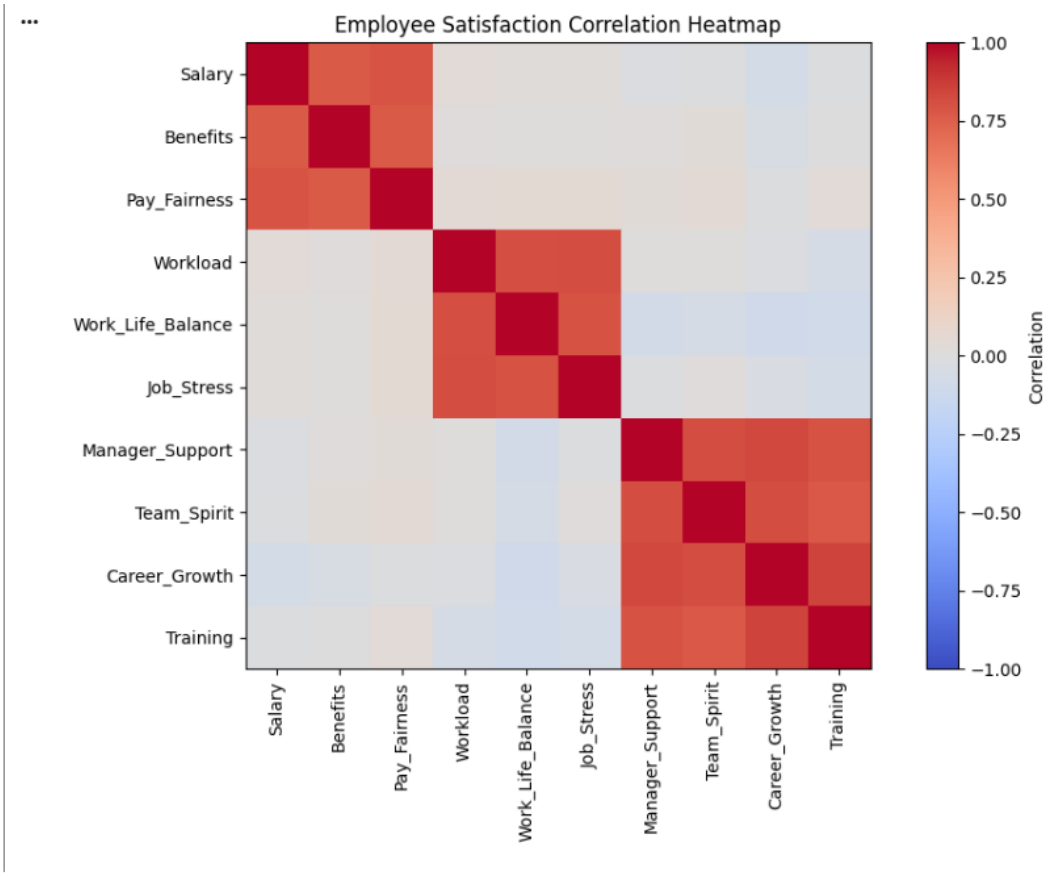
np.random.seed(42)

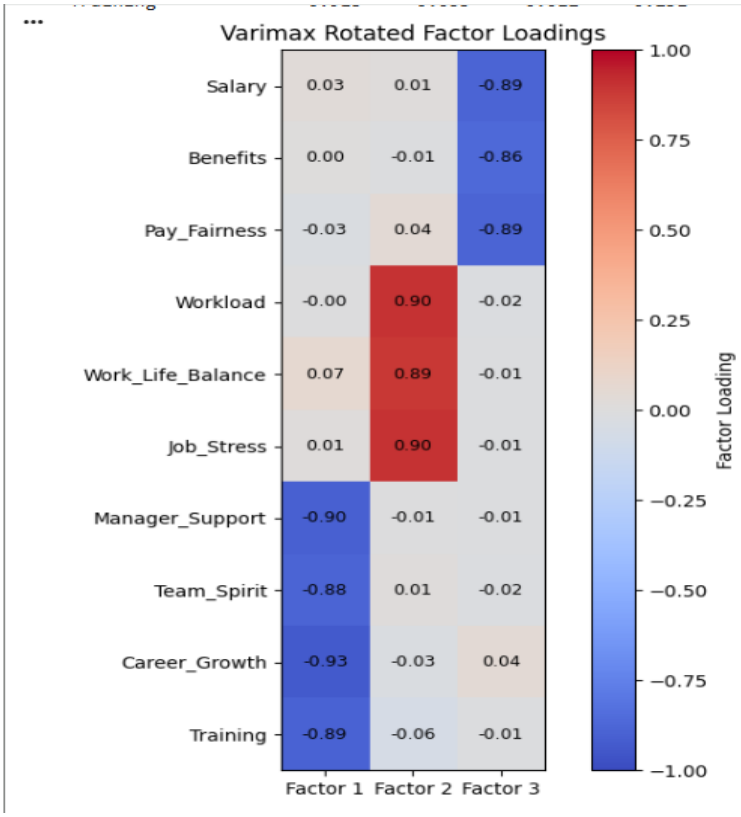
questions = [
```

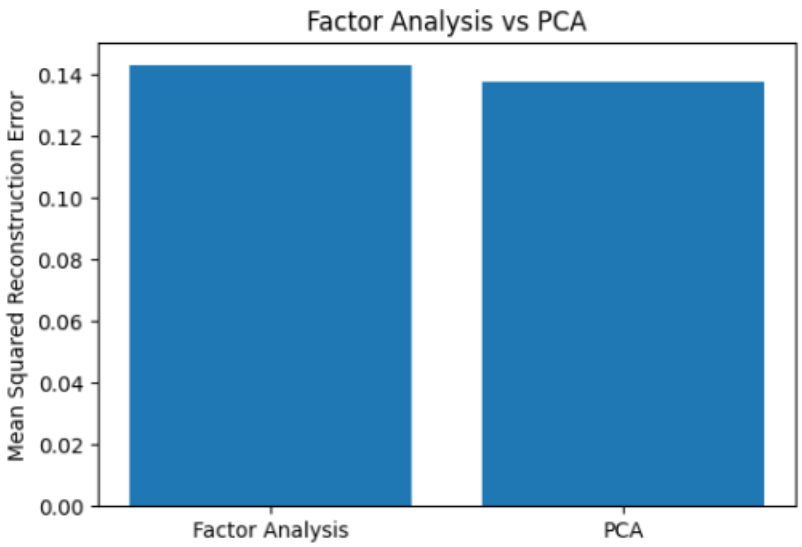
```
    "Salary", "Benefits", "Pay",
    "Workload", "Balance", "Stress",
    "Manager", "Team", "Growth", "Training"
]
# Generate 300 employee responses
latent = np.random.randn(300, 3)
W = np.zeros((10, 3))
W[:3, 0] = [0.9, 0.8, 0.85]
W[3:6, 1] = [0.9, 0.8, 0.85]
W[6:, 2] = [0.9, 0.8, 0.85, 0.75]
data = latent @ W.T + np.random.randn(300, 10) * 0.3
data = np.clip(np.rint(data + 3), 1, 5)
df = pd.DataFrame(data, columns=questions)
print("Survey Data:")
print(df.head())
print("Shape:", df.shape)
# Correlation heatmap
plt.imshow(df.corr(), cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(10), questions, rotation=90)
plt.yticks(range(10), questions)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
X = StandardScaler().fit_transform(df)
```

```
# Factor Analysis
for k in [2, 3, 4]:
    model = FactorAnalysis(
        n_components=k, rotation="varimax", random_state=42
    )
    model.fit(X)
    print(f"\n{k} Factors:")
    print(pd.DataFrame(model.components_.T, index=questions).round(2))
fa = FactorAnalysis(n_components=3, rotation="varimax", random_state=42)
fa.fit(X)
loadings = fa.components_.T
plt.imshow(loadings, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(3), ["F1", "F2", "F3"])
plt.yticks(range(10), questions)
plt.title("Varimax Factor Loadings")
plt.tight_layout()
plt.show()
for i in range(3):
    top = np.argsort(abs(loadings[:, i]))[-3:][::-1]
    print("Factor", i + 1, ":", [questions[j] for j in top])
pca = PCA(n_components=3).fit(X)
fa_recon = fa.transform(X) @ fa.components_ + fa.mean_
pca_recon = pca.inverse_transform(pca.transform(X))
errors = [
```

```
    np.mean((X - fa_recon)**2),
    np.mean((X - pca_recon)**2)
]
print("PCA Explained Variance:", pca.explained_variance_ratio_)
print("Reconstruction Errors:", errors)
plt.bar(["Factor Analysis", "PCA"], errors)
plt.title("FA vs PCA Reconstruction Error")
plt.ylabel("MSE")
plt.show()
```







RESULT

Thus, Factor Analysis was implemented on employee-satisfaction data. The hidden factors were identified using varimax rotation and compared with PCA.

EXPERIMENT 22: INDEPENDENT COMPONENT ANALYSIS AND COMPONENT VISUALIZATION

AIM

To separate mixed signals into independent source signals using Independent Component Analysis (ICA) and compare the results with PCA.

ALGORITHM

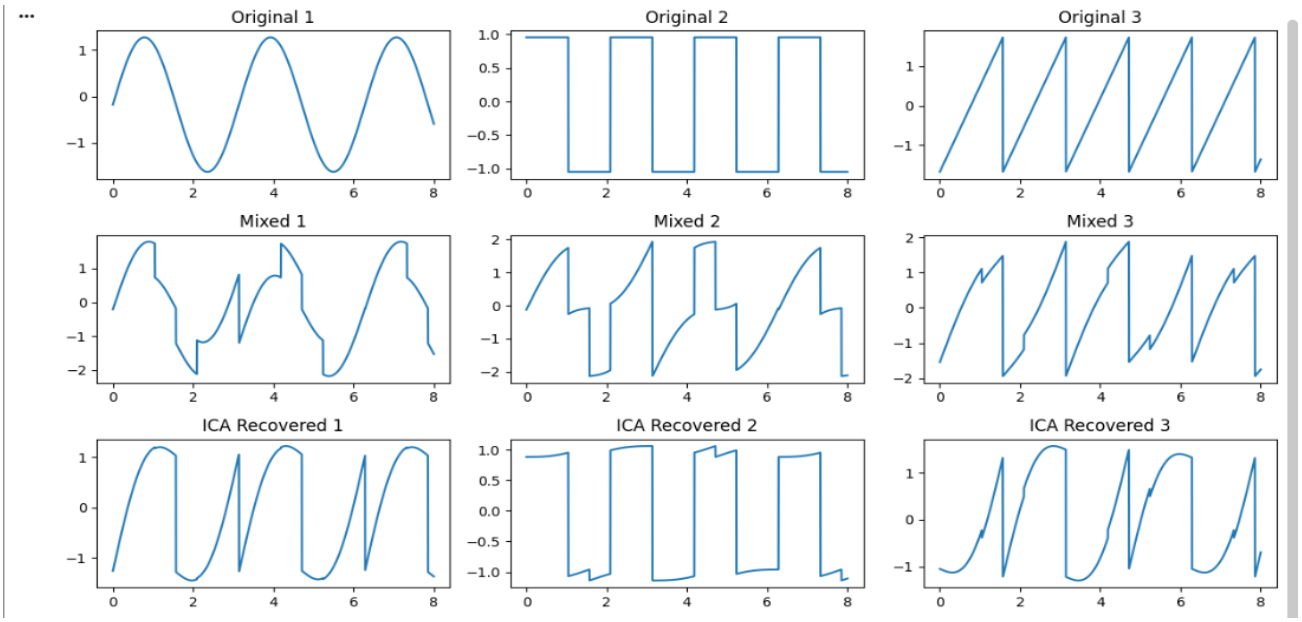
1. Generate sine, square, and sawtooth signals.
2. Mix the signals using a mixing matrix.
3. Apply FastICA to recover independent components.
4. Apply PCA for comparison.
5. Plot original, mixed, ICA-recovered, and PCA-transformed signals.
6. Calculate correlation to evaluate recovery quality.

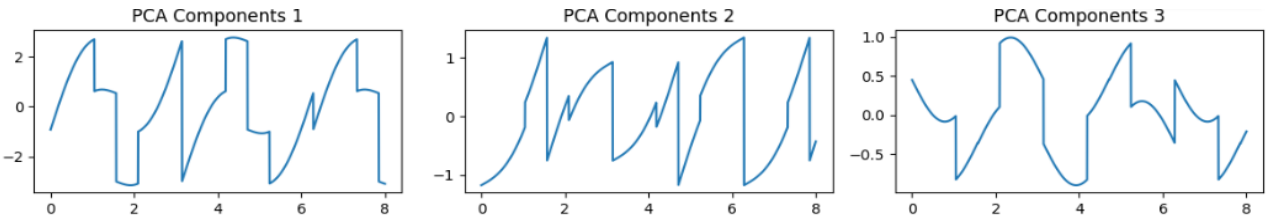
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import sawtooth, square
from sklearn.decomposition import FastICA, PCA
from scipy.optimize import linear_sum_assignment
np.random.seed(42)
t = np.linspace(0, 8, 2000)
# Original source signals
S = np.c_[
    np.sin(2 * t),
    square(3 * t),
    sawtooth(4 * t)
```

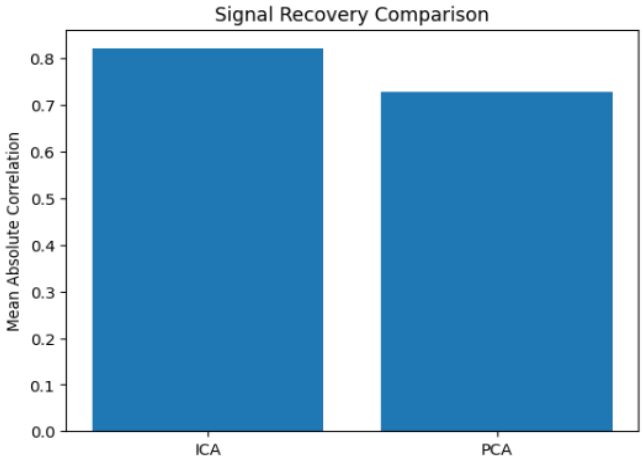
```
]
S = (S - S.mean(axis=0)) / S.std(axis=0)
# Mix signals
A = np.array([
    [1, 0.5, 0.3],
    [0.4, 1, 0.6],
    [0.3, 0.2, 1]
])
X = S @ A.T
# ICA
ica = FastICA(n_components=3, whiten="unit-variance",
              random_state=42, max_iter=2000)
S_ica = ica.fit_transform(X)
pca = PCA(n_components=3)
S_pca = pca.fit_transform(X)
# Match ICA components to sources
corr = np.corrcoef(S.T, S_ica.T)[:3, 3:]
rows, cols = linear_sum_assignment(-abs(corr)z
S_matched = np.zeros_like(S_ica)
for r, c in zip(rows, cols):
    S_matched[:, r] = S_ica[:, c] * np.sign(corr[r, c])
print("ICA Correlation:", np.round(
    [abs(corr[r, c]) for r, c in zip(rows, cols)], 3
))
# Plot all signals
```

```
fig, axes = plt.subplots(4, 3, figsize=(12, 9))
datasets = [S, X, S_matched, S_pca]
titles = ["Original", "Mixed", "ICA Recovered", "PCA Components"]
for i in range(4):
    for j in range(3):
        axes[i, j].plot(t, datasets[i][:, j])
        axes[i, j].set_title(f"{titles[i]} {j+1}")
plt.tight_layout()
plt.show()
# Recovery correlation comparison
def best_correlations(original, recovered):
    c = np.corrcoef(original.T, recovered.T)[:3, 3:]
    r, col = linear_sum_assignment(-abs(c))
    return abs(c[r, col])
ica_corr = best_correlations(S, S_ica)
pca_corr = best_correlations(S, S_pca)
print("ICA Mean Correlation:", np.mean(ica_corr))
print("PCA Mean Correlation:", np.mean(pca_corr))
plt.bar(["ICA", "PCA"], [np.mean(ica_corr), np.mean(pca_corr)])
plt.ylabel("Mean Absolute Correlation")
plt.title("Signal Recovery Comparison")
plt.show()
```






```
ICA Mean Correlation: 0.8208518293949841
PCA Mean Correlation: 0.728319671673627
```



RESULT

Thus, ICA was implemented to separate mixed signals into independent components. The recovered signals were compared with the original signals using correlation, and ICA and PCA performance were visualised.

EXPERIMENT 23: BAYESIAN NETWORK CONSTRUCTION AND VISUALIZATION

AIM

To construct a Bayesian Network for medical diagnosis, define conditional probability distributions, validate the model, and visualise the relationships between variables.

ALGORITHM

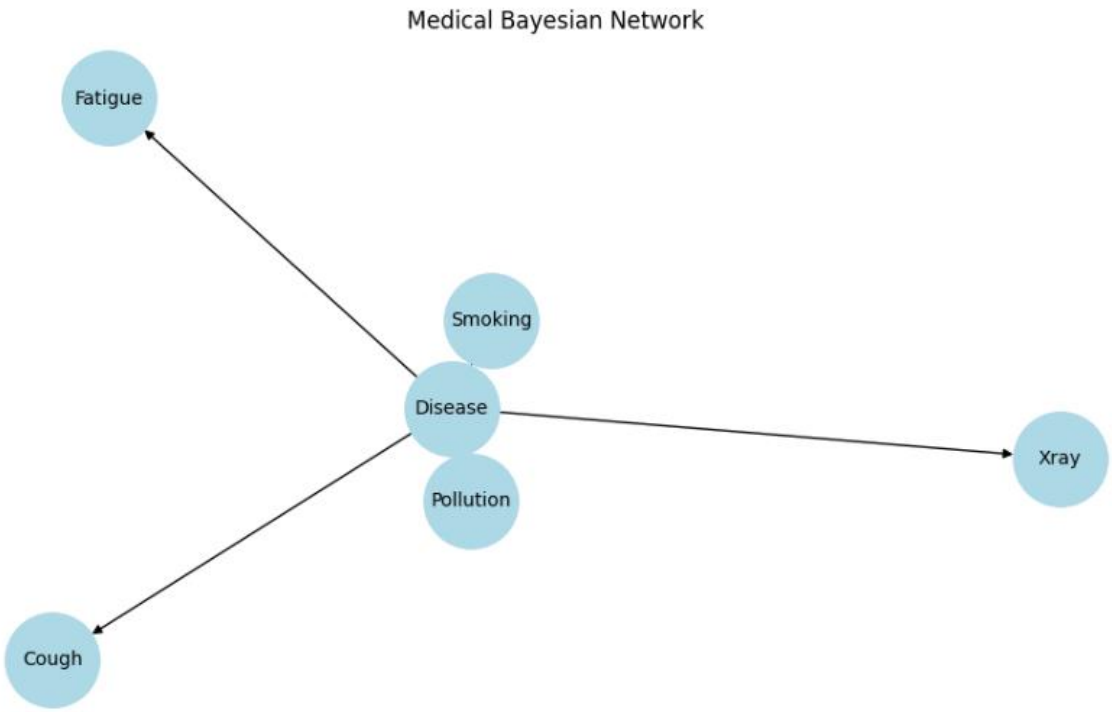
1. Import pgmpy, NetworkX, and Matplotlib.
2. Create six nodes: Smoking, Pollution, Lung Disease, Cough, X-ray, and Fatigue.
3. Connect the nodes using directed edges.
4. Define conditional probability distributions (CPDs).
5. Validate the Bayesian Network.
6. Visualise the network.
7. Display conditional independencies and active trails.

Code:

```
import matplotlib.pyplot as plt
import networkx as nx
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
edges = [
    ("Smoking", "Disease"),
    ("Pollution", "Disease"),
    ("Disease", "Cough"),
    ("Disease", "Xray"),
    ("Disease", "Fatigue")
]
```

```
model = DiscreteBayesianNetwork(edges)
smoking = TabularCPD("Smoking", 2, [[0.7], [0.3]])
pollution = TabularCPD("Pollution", 2, [[0.6], [0.4]])
disease = TabularCPD(
    "Disease", 2,
    [[0.99, 0.90, 0.85, 0.60],
     [0.01, 0.10, 0.15, 0.40]],
    evidence=["Smoking", "Pollution"],
    evidence_card=[2, 2]
)
cough = TabularCPD(
    "Cough", 2,
    [[0.9, 0.2], [0.1, 0.8]],
    evidence=["Disease"],
    evidence_card=[2]
)
    [[0.95, 0.1], [0.05, 0.9]],
    evidence=["Disease"],
    evidence_card=[2]
)
fatigue = TabularCPD(
    "Fatigue", 2,
    [[0.8, 0.3], [0.2, 0.7]],
    evidence=["Disease"],
    evidence_card=[2]
```

```
)
model.add_cpds(smoking, pollution, disease, cough, xray, fatigue)
print("Model Valid:", model.check_model())
print("\nConditional Probability Tables:")
for cpd in model.get_cpds():
    print(cpd)
G = nx.DiGraph(edges)
plt.figure(figsize=(9, 5))
nx.draw(
    G,
    with_labels=True,
    node_color="lightblue",
    node_size=2500,
    font_size=10,
    arrows=True
)
plt.title("Medical Bayesian Network")
plt.show()
print("\nConditional Independencies:")
print(model.get_independencies())
print("\nActive Trails from Smoking:"
```



RESULT

Thus, a medical Bayesian Network was constructed using six variables. Conditional probability tables were defined, the model was validated, and its structure and conditional independencies were displayed.

EXPERIMENT 24: GRAPHICAL MODEL SAMPLING AND PROBABILISTIC INFERENCE

AIM

To perform exact and approximate probabilistic inference using a Bayesian Network and compare the results.

ALGORITHM

1. Construct the medical Bayesian Network.
2. Define the probability distributions.
3. Perform exact inference using Variable Elimination.
4. Generate samples using forward sampling and rejection sampling.
5. Estimate probabilities from the generated samples.

6. Compare exact and approximate probabilities.
7. Plot convergence against sample size.

Code:

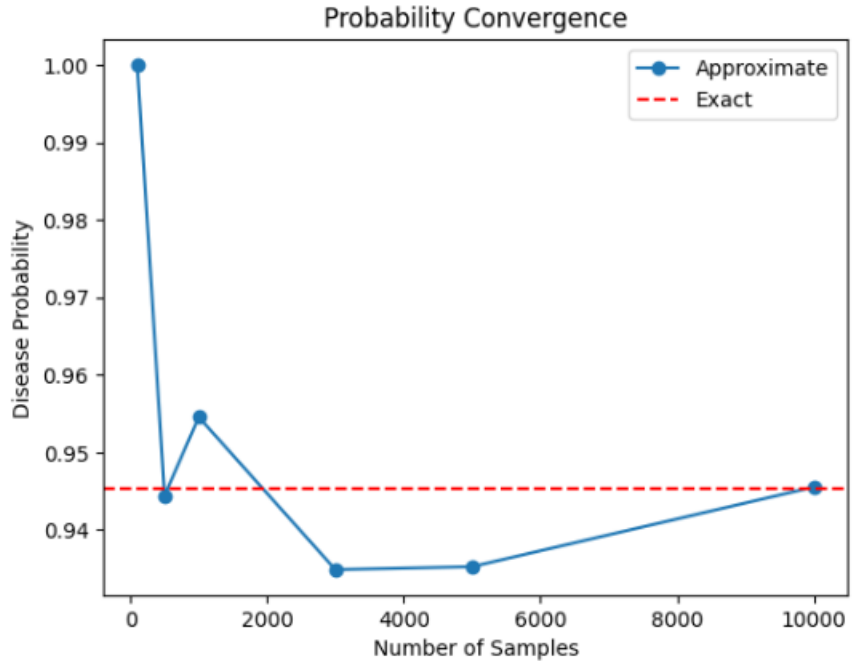
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pgmpy.inference import VariableElimination
from pgmpy.sampling import BayesianModelSampling
from pgmpy.factors.discrete import State
# Exact inference
inference = VariableElimination(model)
evidence = {"Cough": 1, "Xray": 1}
exact = inference.query(
    variables=["Disease"],
    evidence=evidence,
    show_progress=False
)
exact_prob = float(exact.values[1])
print("Exact P(Disease=1 | Cough=1, Xray=1):", exact_prob)
# Check effect of smoking
for status in [0, 1]:
    result = inference.query(
        variables=["Disease"],
        evidence={**evidence, "Smoking": status},
        show_progress=False
```

```
    )
    print("Smoking =", status,
          "Disease Probability =", result.values[1])
# Generate samples
sampler = BayesianModelSampling(model)
forward = sampler.forward_sample(
    size=10000,
    show_progress=False
)
print("\nForward Samples:")
print(forward.head())
rejected = sampler.rejection_sample(
    evidence=[State("Cough", 1), State("Xray", 1)],
    size=1000,
    show_progress=False
)
# Approximate inference using forward samples
sizes = [100, 500, 1000, 3000, 5000, 10000]
estimates = []
for n in sizes:
    sample = forward.iloc[:n]
    filtered = sample[
        (sample["Cough"] == 1) &
        (sample["Xray"] == 1)
    ]
```

```
    probability = (
        filtered["Disease"].mean()
        if len(filtered) else np.nan
    )
    estimates.append(probability)
plt.plot(sizes, estimates, "o-", label="Approximate")
plt.axhline(exact_prob, color="red",
            linestyle="--", label="Exact")
plt.xlabel("Number of Samples")
plt.ylabel("Disease Probability")
plt.title("Probability Convergence")
plt.legend()
plt.show()
comparison = pd.DataFrame({
    "Method": ["Exact", "Forward Sampling", "Rejection Sampling"],
    "Probability": [
        exact_prob,
        estimates[-1],
        rejected["Disease"].mean()
    ]
})
print("\nComparison:")
print(comparison)
```

```
Forward Samples:
   Smoking  Disease  Pollution  Cough  Xray  Fatigue
0        1        0          1      0     0        0
1        0        0          0      0     0        0
2        0        0          0      0     0        0
3        0        0          0      0     0        0
4        0        0          0      0     0        0

Rejection Samples:
   Smoking  Disease  Pollution  Cough  Xray  Fatigue
0        0        1          1      1     1        1
1        1        1          1      1     1        1
2        0        1          0      1     1        0
3        1        1          1      1     1        1
4        1        1          1      1     1        1
```



RESULT

Thus, exact inference was performed using Variable Elimination, and approximate inference was performed using forward and rejection sampling.

EXPERIMENT 25: RESTRICTED BOLTZMANN MACHINE ENERGY AND RECONSTRUCTION

AIM

To implement a Restricted Boltzmann Machine (RBM), calculate its energy, train it using one-step Contrastive Divergence, and reconstruct input data.

ALGORITHM

1. Import NumPy and Matplotlib.

2. Load binary image data.
3. Initialise the RBM weights and biases.
4. Calculate hidden and visible activation probabilities.
5. Train using Contrastive Divergence (CD-1).
6. Calculate energy and reconstruction error.
7. Plot training curves.
8. Display original and reconstructed images.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
np.random.seed(42)
# Load binary digit images
digits = load_digits()
X = (digits.data / 16.0 > 0.5).astype(float)
n_visible = 64
n_hidden = 32

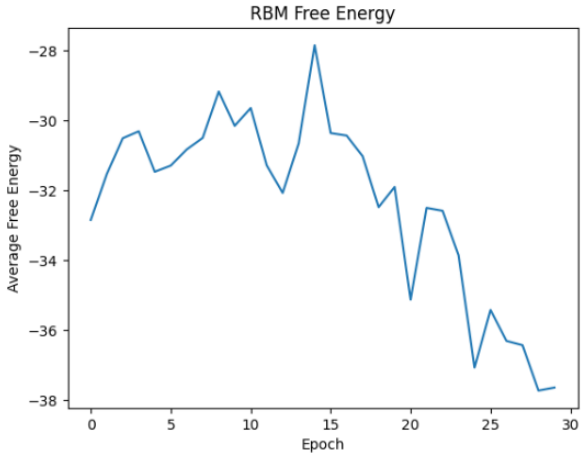

W = np.random.randn(n_visible, n_hidden) * 0.01
a = np.zeros(n_visible)
b = np.zeros(n_hidden)
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -30, 30)))
def energy(v, h):
    return -v @ a - h @ b - v @ W @ h
```

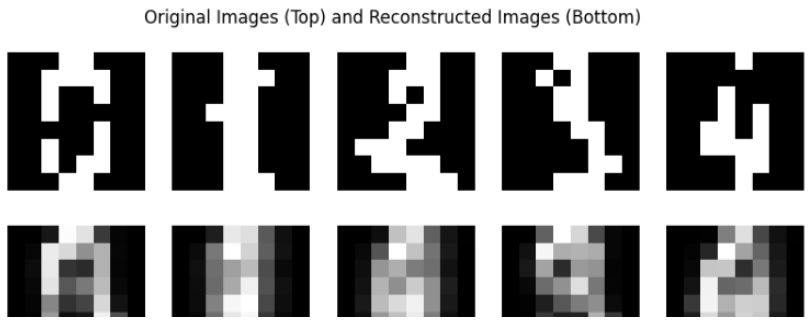
```
def free_energy(v):
    x = v @ W + b
    return -v @ a - np.logaddexp(0, x).sum(axis=1)
epochs = 30
lr = 0.05
batch_size = 64
errors = []
energies = []
for epoch in range(epochs):
    order = np.random.permutation(len(X))
    epoch_error = []
    for start in range(0, len(X), batch_size):
        v0 = X[order[start:start + batch_size]]
        ph0 = sigmoid(v0 @ W + b)
        h0 = (np.random.rand(*ph0.shape) < ph0).astype(float)
        pv1 = sigmoid(h0 @ W.T + a)
        v1 = (np.random.rand(*pv1.shape) < pv1).astype(float)
        ph1 = sigmoid(v1 @ W + b)
        m = len(v0)
        W += lr * (v0.T @ ph0 - v1.T @ ph1) / m
        a += lr * np.mean(v0 - v1, axis=0)
        b += lr * np.mean(ph0 - ph1, axis=0)

print("Final Average Free Energy:", energies[-1])
print("Sample Energy:", energy(X[0], (sigmoid(X[0] @ W + b) > 0.5).astype(float)))
```

```
plt.plot(errors)
plt.xlabel("Epoch")
plt.ylabel("Reconstruction0 Error")
plt.title("RBM Reconstruction Error")
plt.show()
plt.plot(energies)
plt.xlabel("Epoch")
plt.ylabel("Average Free Energy")
plt.title("RBM Free Energy")
plt.show()
original = X[:5]
hidden = sigmoid(original @ W + b)
reconstructed = sigmoid(hidden @ W.T + a)
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i in range(5):
    axes[0, i].imshow(original[i].reshape(8, 8), cmap="gray")
    axes[1, i].imshow(reconstructed[i].reshape(8, 8), cmap="gray")
```







RESULT

Thus, an RBM was implemented and trained using CD-1. The model's energy and reconstruction error were calculated, and the original and reconstructed digit images were visualised.

EXPERIMENT 26: GIBBS SAMPLING AND CONTRASTIVE DIVERGENCE ANALYSIS

AIM

To implement Gibbs sampling in an RBM and compare CD-1, CD-5, CD-10, and Persistent Contrastive Divergence (PCD).

ALGORITHM

1. Load binary image data.
2. Initialise RBM weights and biases.
3. Implement block Gibbs sampling between visible and hidden layers.
4. Train RBMs using CD-1, CD-5, CD-10, and PCD.
5. Measure reconstruction error, pseudo-log-likelihood, and training time.
6. Generate samples after 1, 10, 100, and 1000 Gibbs steps.
7. Compare the training methods using graphs.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import time
from sklearn.datasets import load_digits
np.random.seed(42)
X = (load_digits().data / 16 > 0.5).astype(float)
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -30, 30)))

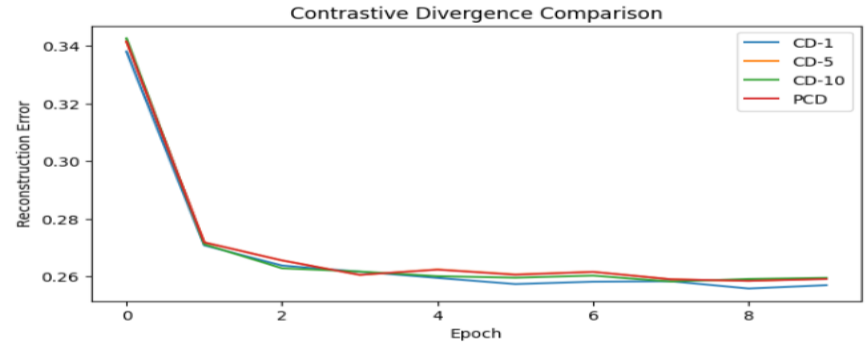

def train_rbm(method, epochs=10):
    rng = np.random.default_rng(42)
```

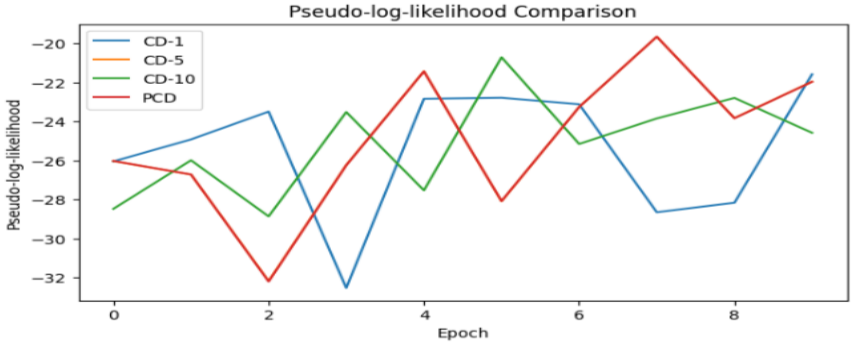
```
W = rng.normal(0, 0.01, (64, 32))
a = np.zeros(64)
b = np.zeros(32)
errors, likelihoods = [], []
persistent = None
start_time = time.time()
def free_energy(v):
    return -v @ a - np.logaddexp(0, v @ W + b).sum(axis=1)
for epoch in range(epochs):
    batch_errors = []
    for start in range(0, len(X) - 64, 64):
        v0 = X[start:start+64]
        ph0 = sigmoid(v0 @ W + b)
        if method == "PCD" and persistent is not None:
            v = persistent.copy()
        else:
            v = v0.copy()
        k = {"CD-1": 1, "CD-5": 5,
             "CD-10": 10, "PCD": 5}[method]
        for _ in range(k):
            ph = sigmoid(v @ W + b)
            h = (rng.random(ph.shape) < ph).astype(float)
            pv = sigmoid(h @ W.T + a)
            v = (rng.random(pv.shape) < pv).astype(float)
        if method == "PCD":
```

```
            persistent = v.copy()
        phk = sigmoid(v @ W + b)
        W += 0.05 * (v0.T @ ph0 - v.T @ phk) / 64
        a += 0.05 * np.mean(v0 - v, axis=0)
        b += 0.05 * np.mean(ph0 - phk, axis=0
        batch_errors.append(np.mean((v0 - v)**2))
    errors.append(np.mean(batch_errors))
    # Pseudo-log-likelihood by flipping one visible bit
    sample = X[:100].copy()
    flipped = sample.copy()
    indices = rng.integers(0, 64, len(sample))
    flipped[np.arange(len(sample)), indices] = (
        1 - flipped[np.arange(len(sample)), indices]
    )
    delta = free_energy(flipped) - free_energy(sample)
    pll = 64 * np.mean(-np.logaddexp(0, -delta))
    likelihoods.append(pll)
return W, a, b, errors, likelihoods, time.time()-start_time
results = {}
for method in ["CD-1", "CD-5", "CD-10", "PCD"]:
    results[method] = train_rbm(method)
    print(method, "Time:", round(results[method][5], 2),
          "Error:", round(results[method][3][-1], 4))
plt.figure(figsize=(8, 4))
for method, result in results.items():
```

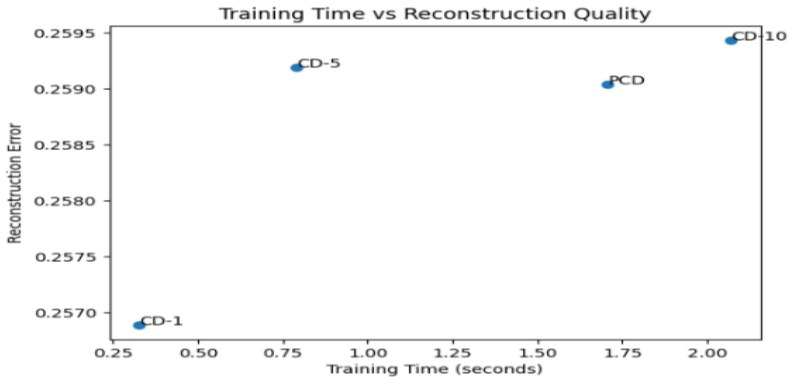
```
    plt.plot(result[3], label=method)
plt.xlabel("Epoch")
plt.ylabel("Reconstruction Error")
plt.title("Contrastive Divergence Comparison")
plt.legend()
plt.show()
plt.figure(figsize=(8, 4))
for method, result in results.items():
    plt.plot(result[4], label=method)
plt.xlabel("Epoch")
plt.ylabel("Pseudo-log-likelihood")
plt.title("Pseudo-log-likelihood Comparison")
plt.legend()
plt.show()
# Gibbs sampling
W, a, b = results["CD-5"][:3]
rng = np.random.default_rng(123)
v = rng.integers(0, 2, (1, 64)).astype(float)
fig, axes = plt.subplots(1, 4, figsize=(10, 3))
previous = 0
for ax, steps in zip(axes, [1, 10, 100, 1000]):
    for _ in range(steps - previous):
        ph = sigmoid(v @ W + b)
        h = (rng.random(ph.shape) < ph).astype(float)
        pv = sigmoid(h @ W.T + a)
```

```
        v = (rng.random(pv.shape) < pv).astype(float)
    ax.imshow(v.reshape(8, 8), cmap="gray")
    ax.set_title(f"{steps} Steps")
    ax.axis("off")
    previous = steps
plt.suptitle("Gibbs Sampling")
plt.show()
methods = list(results)
times = [results[m][5] for m in methods]
errors = [results[m][3][-1] for m in methods]
plt.scatter(times, errors)
for i, method in enumerate(methods):
    plt.annotate(method, (times[i], errors[i]))
plt.xlabel("Training Time (seconds)")
plt.ylabel("Reconstruction Error")
plt.title("Training Time vs Reconstruction Quality")
plt.show()
```









RESULT

Thus, Gibbs sampling and Contrastive Divergence were implemented. CD-1, CD-5, CD-10, and PCD were compared using reconstruction error, pseudo-log-likelihood, and training time. Generated samples were visualised at different Gibbs steps.

EXPERIMENT 27: DEEP BELIEF NETWORK REPRESENTATION LEARNING

AIM

To implement a Deep Belief Network (DBN) using stacked RBMs and compare pretrained and randomly initialised neural networks for handwritten digit classification.

ALGORITHM

1. Load the MNIST handwritten digit dataset.
2. Normalise the images and prepare 1,000 labelled training examples.
3. Pretrain two stacked RBMs using unlabelled images.
4. Visualise the weights learned by the first RBM.
5. Transfer the pretrained weights into a Keras neural network.
6. Fine-tune the model using labelled images.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from sklearn.neural_network import BernoulliRBM

from sklearn.decomposition import PCA

from tensorflow.keras.datasets import mnist

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Input, Dense

np.random.seed(42)

tf.random.set_seed(42)

# Load MNIST

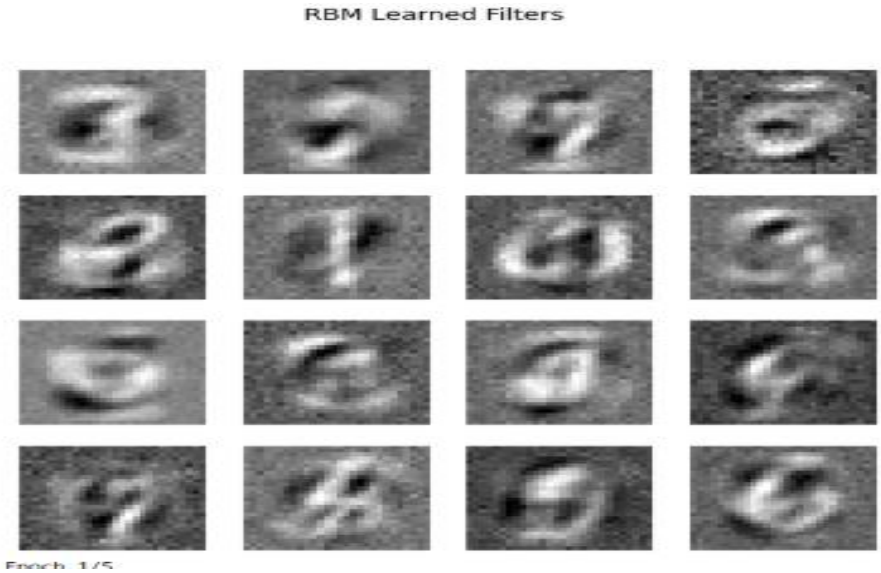
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

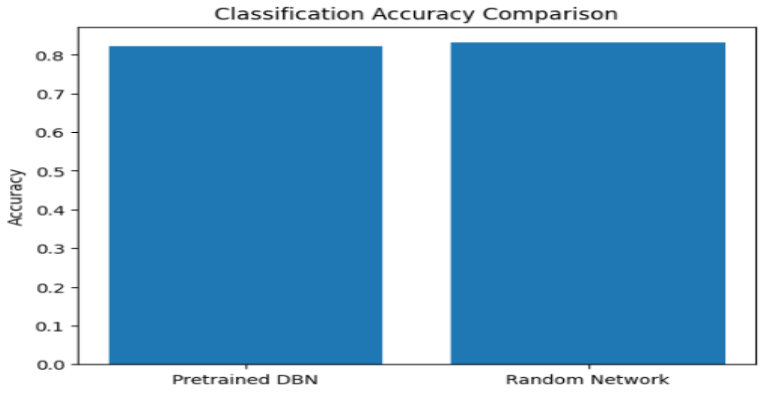
```
x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0
# Unlabelled data for pretraining
unlabelled = x_train[:5000]
unlabelled_binary = (unlabelled > 0.5).astype(float)
# RBM Layer 1
print("Training RBM 1...")
rbm1 = BernoulliRBM(
    n_components=500,
    learning_rate=0.05,
    batch_size=100,
    n_iter=5,
    random_state=42
)
rbm1.fit(unlabelled_binary)
# RBM Layer 2
print("Training RBM 2...")
features1 = rbm1.transform(unlabelled_binary)
rbm2 = BernoulliRBM(
    n_components=250,
    learning_rate=0.05,
    batch_size=100,
    n_iter=5,
    random_state=42
)
```

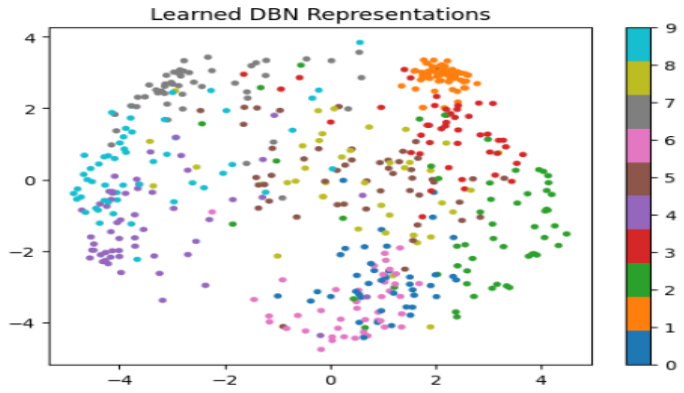
```
rbm2.fit(features1)
# Visualise first-layer filters
fig, axes = plt.subplots(4, 4, figsize=(7, 7))
for i, ax in enumerate(axes.flat):
    ax.imshow(rbm1.components_[i].reshape(28, 28),
              cmap="gray")
    ax.axis("off")
plt.suptitle("RBM Learned Filters")
plt.show()
# Only 1000 labelled examples
x_labeled = x_train[5000:6000]
y_labeled = y_train[5000:6000]
def create_model(pretrained=False):
    model = Sequential([
        Input(shape=(784,)),
        Dense(500, activation="sigmoid"),
        Dense(250, activation="sigmoid"),
        Dense(10, activation="softmax")
    ])
    if pretrained:
        model.layers[0].set_weights([
            rbm1.components_.T.astype("float32"),
            rbm1.intercept_hidden_.astype("float32")
        ])
        model.layers[1].set_weights([
```

```
            rbm2.components_.T.astype("float32"),
            rbm2.intercept_hidden_.astype("float32")
        ])
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Train pretrained network
pretrained_model = create_model(True)
pretrained_model.fit(
    x_labeled, y_labeled,
    epochs=5, batch_size=32, verbose=1
)
# Train randomly initialized network
random_model = create_model(False)
random_model.fit(
    x_labeled, y_labeled,
    epochs=5, batch_size=32, verbose=1
)
# Accuracy comparison
acc1 = pretrained_model.evaluate(
    x_test[:2000], y_test[:2000], verbose=0
)[1]
```

```
acc2 = random_model.evaluate(
    x_test[:2000], y_test[:2000], verbose=0
)[1]
print("Pretrained DBN Accuracy:", acc1)
print("Random Network Accuracy:", acc2)
plt.show()
# Visualize representations
features = feature_model.predict(x_test[:500], verbose=0)
pca = PCA(n_components=2)
features_2d = pca.fit_transform(features)
plt.scatter(
    features_2d[:, 0],
    features_2d[:, 1],
    c=y_test[:500],
    cmap="tab10",
    s=10
)
plt.colorbar()
plt.title("Learned DBN Representations")
plt.show()
```







RESULT

Thus, a Deep Belief Network was implemented using two stacked RBMs. The pretrained model was fine-tuned for digit classification and compared with a randomly initialised network. Learned filters, classification accuracy, and two-dimensional feature representations were displayed.

EXPERIMENT 28: DEEP BOLTZMANN MACHINE SIMULATION

AIM

To simulate a two-hidden-layer Deep Boltzmann Machine (DBM), perform approximate learning using mean-field inference and Gibbs sampling, and compare reconstruction quality with a single-layer RBM.

ALGORITHM

1. Load grayscale handwritten digit images.
2. Convert images into binary data.
3. Initialise the visible and two hidden layers of a DBM.
4. Estimate hidden-unit probabilities using mean-field iterations.
5. Use Gibbs sampling to obtain negative-phase samples.
6. Update the model parameters.
7. Visualise learned weights and hidden activations.
8. Generate and reconstruct images.
9. Compare reconstruction error with a single-layer RBM.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.neural_network import BernoulliRBM

np.random.seed(42)

rng = np.random.default_rng(42)

X = (load_digits().data / 16 > 0.5).astype(float)

X = X[:1000]

nv, nh1, nh2 = 64, 32, 16

W1 = rng.normal(0, 0.02, (nv, nh1))
```

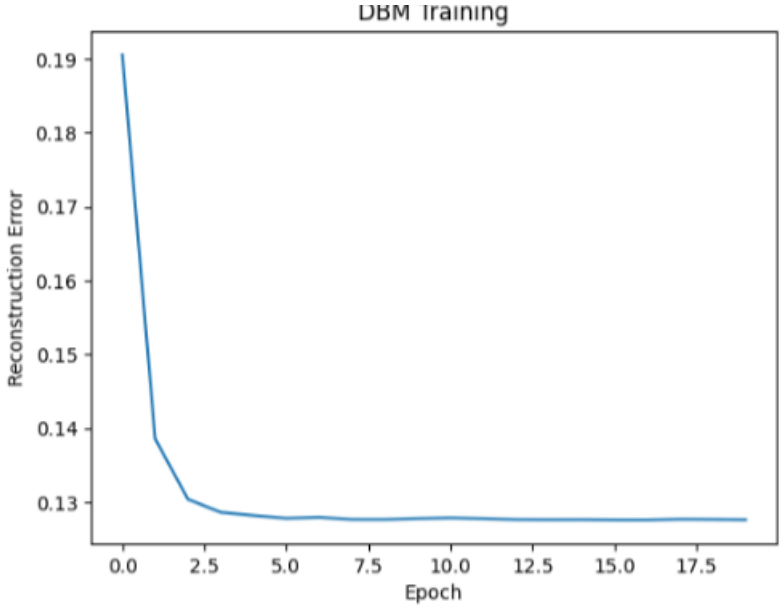
```
W2 = rng.normal(0, 0.02, (nh1, nh2))
a = np.zeros(nv)
b1 = np.zeros(nh1)
b2 = np.zeros(nh2)
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -30, 30)))
def mean_field(v, steps=5):
    h1 = np.full((len(v), nh1), 0.5)
    h2 = np.full((len(v), nh2), 0.5)
    for _ in range(steps):
        h1 = sigmoid(v @ W1 + h2 @ W2.T + b1)
        h2 = sigmoid(h1 @ W2 + b2)
    return h1, h2
errors = []
lr = 0.02
# Approximate DBM training
for epoch in range(20):
    order = rng.permutation(len(X))
    batch_errors = []
    for start in range(0, len(X), 50):
        v0 = X[order[start:start+50]]
        m = len(v0)
        # Positive phase: mean-field inference
        h1_pos, h2_pos = mean_field(v0)
        # Negative phase: block Gibbs sampling
```

```
        v = v0.copy()
        h1 = h1_pos.copy()
        h2 = h2_pos.copy()
        for _ in range(3):
            p1 = sigmoid(v @ W1 + h2 @ W2.T + b1)
            h1 = (rng.random(p1.shape) < p1).astype(float)
            p2 = sigmoid(h1 @ W2 + b2)
            h2 = (rng.random(p2.shape) < p2).astype(float)
            pv = sigmoid(h1 @ W1.T + a)
            v = (rng.random(pv.shape) < pv).astype(float)
        h1_neg, h2_neg = mean_field(v)
        W1 += lr * (
            v0.T @ h1_pos - v.T @ h1_neg
        ) / m
        W2 += lr * (
            h1_pos.T @ h2_pos - h1_neg.T @ h2_neg
        ) / m
        a += lr * np.mean(v0 - v, axis=0)
        b1 += lr * np.mean(h1_pos - h1_neg, axis=0)
        b2 += lr * np.mean(h2_pos - h2_neg, axis=0)
        batch_errors.append(np.mean((v0 - pv)**2))
    errors.append(np.mean(batch_errors))
print("Final DBM Training Error:", errors[-1])
plt.plot(errors)
plt.xlabel("Epoch")
```

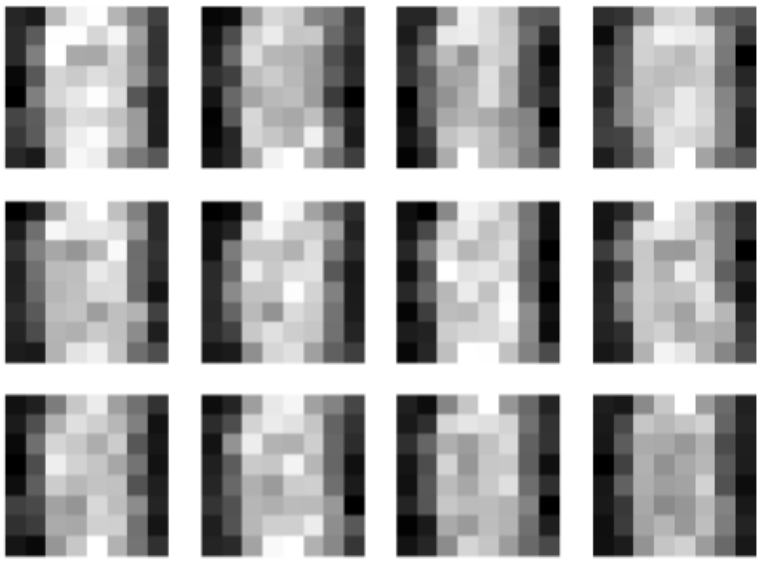
```
plt.ylabel("Reconstruction Error")
plt.title("DBM Training")
plt.show()
# Visualize first-layer filters
fig, axes = plt.subplots(4, 4, figsize=(7, 7))
for i, ax in enumerate(axes.flat):
    ax.imshow(W1[:, i].reshape(8, 8), cmap="gray")
    ax.axis("off")
plt.suptitle("DBM Learned Filters")
plt.show()
# Hidden activations
h1, h2 = mean_field(X[:10])
plt.imshow(h1, cmap="viridis", aspect="auto")
plt.colorbar()
plt.xlabel("Hidden Neuron")
plt.ylabel("Sample")
plt.title("First Hidden Layer Activations")
plt.show()
# Reconstruct samples
original = X[:5]
h1, h2 = mean_field(original)
reconstructed = sigmoid(h1 @ W1.T + a)
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i in range(5):
    axes[0, i].imshow(original[i].reshape(8, 8), cmap="gray")
```

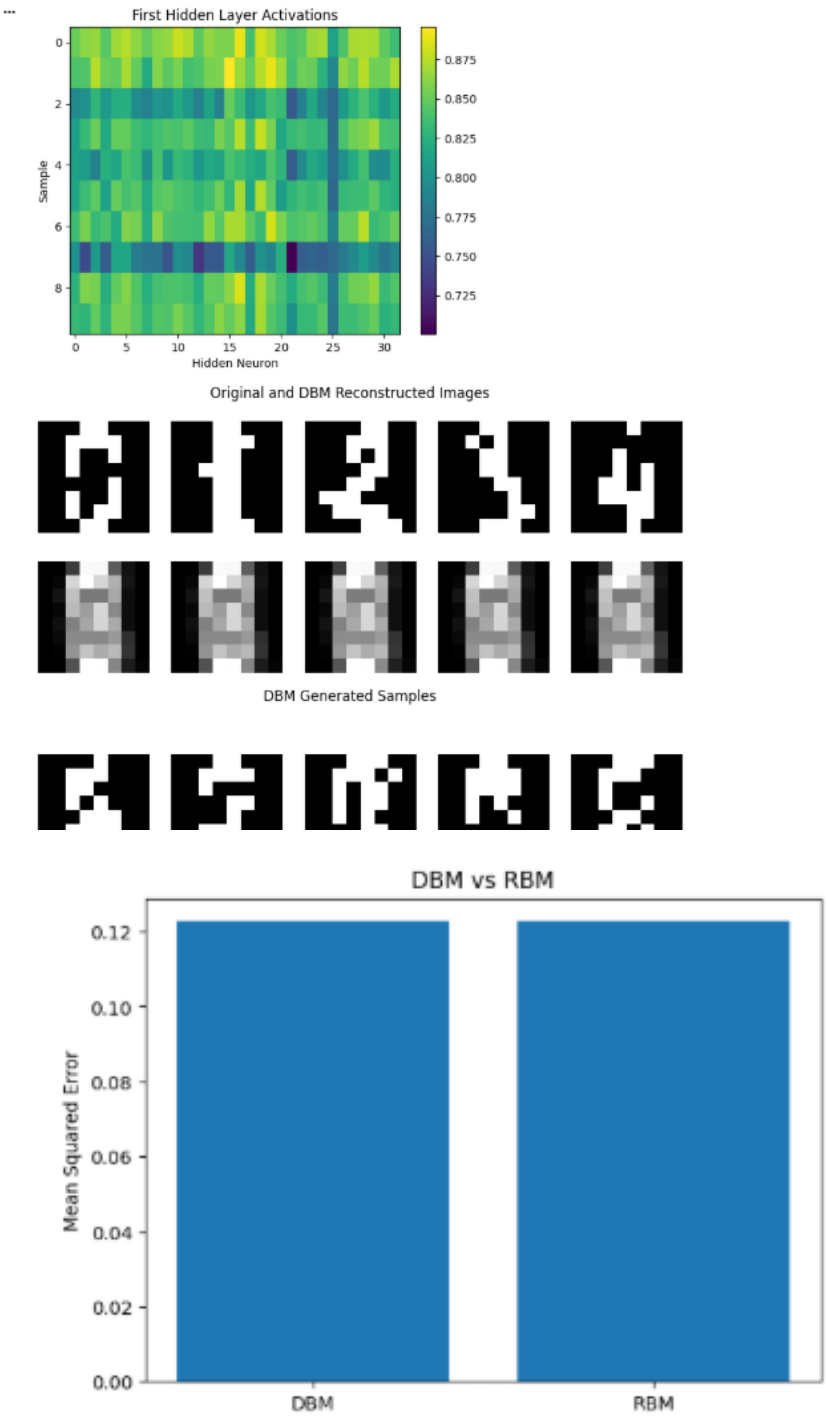
```
    axes[1, i].imshow(reconstructed[i].reshape(8, 8), cmap="gray")
    axes[0, i].axis("off")
    axes[1, i].axis("off")
plt.suptitle("Original and DBM Reconstructed Images")
plt.show()
# Generate samples from the DBM
v = rng.integers(0, 2, (5, nv)).astype(float)
h2 = rng.integers(0, 2, (5, nh2)).astype(float)
for _ in range(500):
    p1 = sigmoid(v @ W1 + h2 @ W2.T + b1)
    h1 = (rng.random(p1.shape) < p1).astype(float)
    p2 = sigmoid(h1 @ W2 + b2)
    h2 = (rng.random(p2.shape) < p2).astype(float)
    pv = sigmoid(h1 @ W1.T + a)
    v = (rng.random(pv.shape) < pv).astype(float)
fig, axes = plt.subplots(1, 5, figsize=(10, 3))
for i, ax in enumerate(axes):
    ax.imshow(v[i].reshape(8, 8), cmap="gray")
    ax.axis("off")
plt.suptitle("DBM Generated Samples")
plt.show()
rbm = BernoulliRBM(
    n_components=32,
    learning_rate=0.02,
    n_iter=20,
```

```
    batch_size=50,
    random_state=42
)
rbm.fit(X)
rbm_hidden = rbm.transform(original)
rbm_recon = sigmoid(
    rbm_hidden @ rbm.components_ + rbm.intercept_visible_
)
dbm_error = np.mean((original - reconstructed)**2)
rbm_error = np.mean((original - rbm_recon)**2)
print("DBM Reconstruction Error:", dbm_error)
print("RBM Reconstruction Error:", rbm_error)
plt.bar(["DBM", "RBM"], [dbm_error, rbm_error])
plt.ylabel("Mean Squared Error")
plt.title("DBM vs RBM")
plt.show()
```



DBM Learned Filters





RESULT

Thus, a two-hidden-layer Deep Boltzmann Machine was simulated using approximate mean-field and Gibbs-sampling updates. Learned filters, hidden activations, generated images, and reconstructed images were displayed. Its reconstruction error was compared with that of a single-layer RBM.

EXPERIMENT 29: VAE/GAN SAMPLE GENERATION

AIM

To implement a Variational Autoencoder (VAE) and a Generative Adversarial Network (GAN) for generating new Fashion-MNIST images.

ALGORITHM

1. Load the Fashion-MNIST dataset.
2. Normalise the images.
3. Build a VAE containing an encoder, reparameterisation layer, and decoder.
4. Train the VAE using reconstruction loss and KL divergence.
5. Generate images by sampling the latent space.
6. Build a DCGAN using a generator and discriminator.
7. Train the GAN by alternating generator and discriminator updates.
8. Save generated samples at different epochs.
9. Plot generator and discriminator loss curves.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.datasets import fashion_mnist
tf.random.set_seed(42)
np.random.seed(42)
# Load Fashion-MNIST
(x_train, _), (x_test, _) = fashion_mnist.load_data()
x_train = x_train[:10000].astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
latent_dim = 2
encoder_input = layers.Input(shape=(28, 28, 1))
x = layers.Flatten()(encoder_input)
x = layers.Dense(128, activation="relu")(x)
z_mean = layers.Dense(latent_dim)(x)
z_log_var = layers.Dense(latent_dim)(x)
encoder = Model(
    encoder_input, [z_mean, z_log_var],
    name="encoder"
)
decoder_input = layers.Input(shape=(latent_dim,))
x = layers.Dense(128, activation="relu")(decoder_input)
x = layers.Dense(784, activation="sigmoid")(x)
decoder_output = layers.Reshape((28, 28, 1))(x)
decoder = Model(decoder_input, decoder_output, name="decoder")
class VAE(Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.loss_tracker = tf.keras.metrics.Mean(name="loss")
    @property
    def metrics(self):
        return [self.loss_tracker]
```

```
def train_step(self, data):
    if isinstance(data, tuple):
        data = data[0]
    with tf.GradientTape() as tape:
        mean, log_var = self.encoder(data)
        epsilon = tf.random.normal(tf.shape(mean))
        z = mean + tf.exp(0.5 * log_var) * epsilon
        reconstruction = self.decoder(z)
        bce = tf.keras.losses.binary_crossentropy(
            data, reconstruction
        )
        reconstruction_loss = tf.reduce_mean(
            tf.reduce_sum(bce, axis=(1, 2))
        )
        kl_loss = -0.5 * tf.reduce_mean(
            tf.reduce_sum(
                1 + log_var - tf.square(mean) - tf.exp(log_var),
                axis=1
            )
        )
        loss = reconstruction_loss + kl_loss
    gradients = tape.gradient(loss, self.trainable_weights)
    self.optimizer.apply_gradients(
        zip(gradients, self.trainable_weights)
    )
```

```
        self.loss_tracker.update_state(loss)
        return {"loss": self.loss_tracker.result()}
vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam(0.001))
print("Training VAE...")
vae_history = vae.fit(
    x_train[..., None],
    epochs=5,
    batch_size=128,
    verbose=1
plt.plot(vae_history.history["loss"])
plt.title("VAE Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
# Generate VAE images
z = np.random.normal(size=(10, latent_dim)).astype("float32")
vae_images = decoder.predict(z, verbose=0)
fig, axes = plt.subplots(1, 10, figsize=(15, 2))
for i, ax in enumerate(axes):
    ax.imshow(vae_images[i, :, :, 0], cmap="gray")
    ax.axis("off")
plt.suptitle("VAE Generated Images")
plt.show()
# 2D latent-space grid
grid = np.linspace(-2, 2, 10)
```

```
z_grid = np.array(
    [[i, j] for i in grid for j in grid],
    dtype="float32"
)
grid_images = decoder.predict(z_grid, verbose=0)
fig, axes = plt.subplots(10, 10, figsize=(10, 10))
for i, ax in enumerate(axes.flat):
    ax.imshow(grid_images[i, :, :, 0], cmap="gray")
    ax.axis("off")
plt.suptitle("VAE Latent Space")
plt.show()
gan_latent_dim = 64
generator = tf.keras.Sequential([
    layers.Input(shape=(gan_latent_dim,)),
    layers.Dense(7 * 7 * 128),
    layers.Reshape((7, 7, 128)),
    layers.BatchNormalization(),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Conv2DTranspose(
        64, 4, strides=2, padding="same"
    ),
    layers.BatchNormalization(),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Conv2DTranspose(
        1, 4, strides=2, padding="same",
```

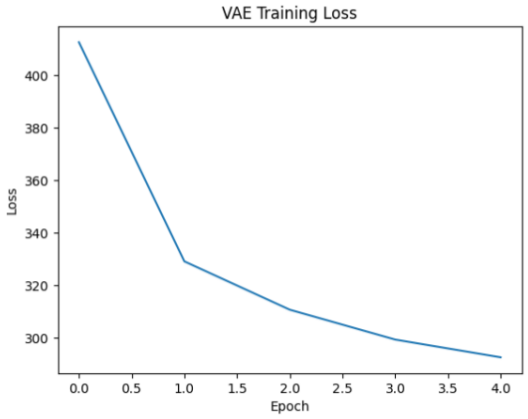
```
        activation="sigmoid"
    )
])
discriminator = tf.keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(64, 4, strides=2, padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Conv2D(128, 4, strides=2, padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Flatten(),
    layers.Dense(1)
])
g_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)
d_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)
loss_fn = tf.keras.losses.BinaryCrossentropy(from_logits=True)
@tf.function
def train_gan_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal((batch_size, gan_latent_dim))

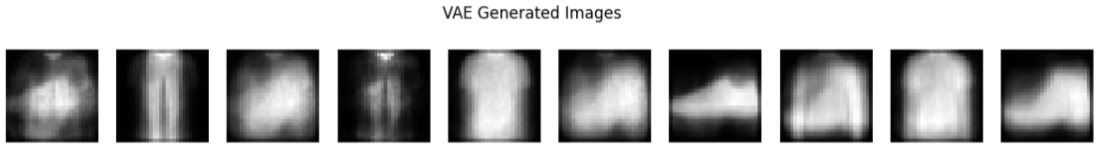
    # Train discriminator
    with tf.GradientTape() as d_tape:
        fake_images = generator(noise, training=True)
        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)
```

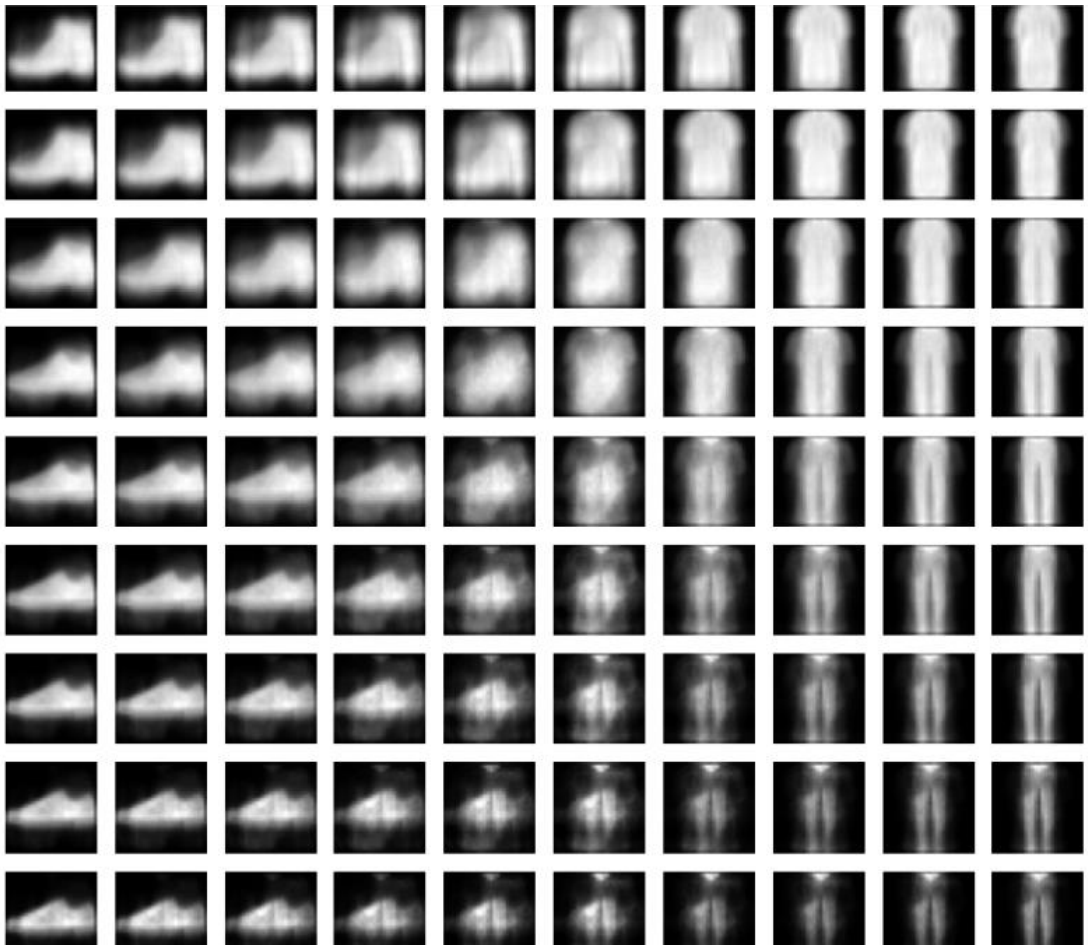
```
    d_loss = (
        loss_fn(tf.ones_like(real_output), real_output) +
        loss_fn(tf.zeros_like(fake_output), fake_output)
    )
d_grads = d_tape.gradient(
    d_loss, discriminator.trainable_variables
)
d_optimizer.apply_gradients(
    zip(d_grads, discriminator.trainable_variables)
)
# Train generator
noise = tf.random.normal((batch_size, gan_latent_dim))
with tf.GradientTape() as g_tape:
    fake_images = generator(noise, training=True)
    fake_output = discriminator(fake_images, training=False)
    g_loss = loss_fn(
        tf.ones_like(fake_output), fake_output
    )
g_grads = g_tape.gradient(
    g_loss, generator.trainable_variables
)
g_optimizer.apply_gradients(
    zip(g_grads, generator.trainable_variables)
)
return d_loss, g_loss
```

```
dataset = (
    tf.data.Dataset.from_tensor_slices(x_train[..., None])
    .shuffle(10000)
    .batch(128)
    .prefetch(tf.data.AUTOTUNE)
)
g_losses = []
d_losses = []
print("Training GAN...")
for epoch in range(5):
    epoch_g = []
    epoch_d = []
    for real_images in dataset:
        d_loss, g_loss = train_gan_step(real_images)
        epoch_d.append(float(d_loss))
        epoch_g.append(float(g_loss))
    d_losses.append(np.mean(epoch_d))
    g_losses.append(np.mean(epoch_g))
    print(
        f"Epoch {epoch+1}: "
        f"D Loss={d_losses[-1]:.4f}, "
        f"G Loss={g_losses[-1]:.4f}"
    )
    noise = tf.random.normal((10, gan_latent_dim))
    samples = generator(noise, training=False).numpy()
```

```
    fig, axes = plt.subplots(1, 10, figsize=(15, 2))
    for i, ax in enumerate(axes):
        ax.imshow(samples[i, :, :, 0], cmap="gray")
        ax.axis("off")
    plt.suptitle(f"GAN Generated Images - Epoch {epoch+1}")
    plt.savefig(f"gan_epoch_{epoch+1}.png")
    plt.show()
plt.plot(g_losses, label="Generator")
plt.plot(d_losses, label="Discriminator")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("GAN Training Loss")
plt.legend()
plt.show(
print("Experiment 29 Completed")
```

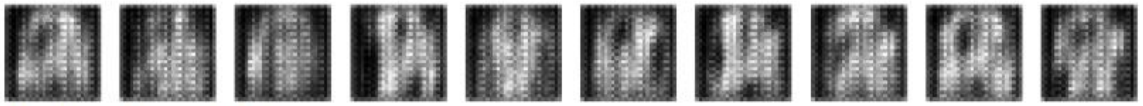






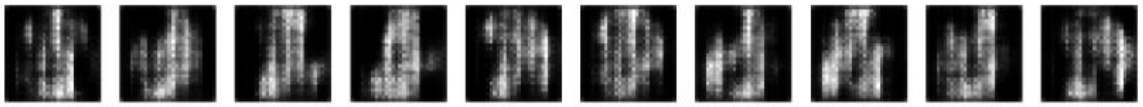
...

GAN Generated Images - Epoch 1



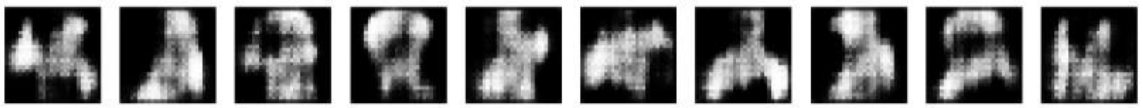
Epoch 2: D Loss=1.0702, G Loss=0.9058

GAN Generated Images - Epoch 2



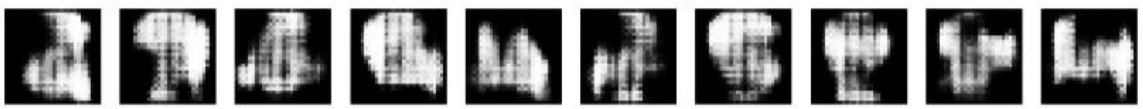
Epoch 3: D Loss=1.0447, G Loss=0.9687

GAN Generated Images - Epoch 3

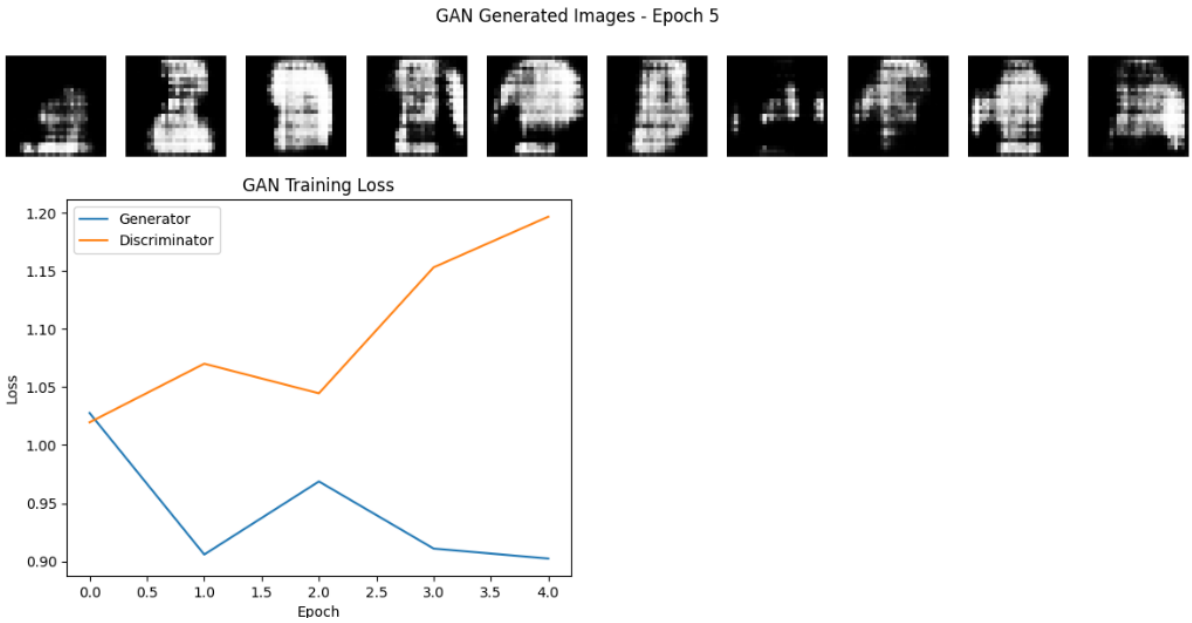


Epoch 4: D Loss=1.1532, G Loss=0.9109

GAN Generated Images - Epoch 4



Epoch 5: D Loss=1.1969, G Loss=0.9023



RESULT

Thus, a Variational Autoencoder and a Deep Convolutional Generative Adversarial Network were implemented using Fashion-MNIST. Both models generated synthetic images, and their training losses were visualised.

EXPERIMENT 30: GENERATIVE MODEL OUTPUT COMPARISON AND EVALUATION

AIM

To evaluate and compare the image quality and diversity of VAE and GAN models using Inception Score, Fréchet Inception Distance (FID), diversity analysis, and PCA visualisation.

ALGORITHM

1. Load the trained VAE and GAN models.
2. Generate 1,000 synthetic images from each model.
3. Train a Fashion-MNIST classifier for feature extraction.
4. Calculate a classifier-based Inception Score.
5. Calculate FID using real and generated feature distributions.

6. Analyse the class distribution of generated images.
7. Visualise real and generated image features using PCA.
8. Compare the models using a Pandas table.
9. Recommend a model based on the measured results.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.datasets import fashion_mnist
from sklearn.decomposition import PCA
from scipy.linalg import sqrtm
np.random.seed(42)
tf.random.set_seed(42)
# Load real images and labels
(_, _), (x_test, y_test) = fashion_mnist.load_data()
x_real = x_test.astype("float32") / 255.0
x_real = x_real[..., None]
# Train a classifier on Fashion-MNIST
(x_cls, y_cls), _ = fashion_mnist.load_data()
x_cls = x_cls[:15000].astype("float32") / 255.0
x_cls = x_cls[..., None]
y_cls = y_cls[:15000]
classifier_input = layers.Input(shape=(28, 28, 1))
```

```
x = layers.Conv2D(32, 3, activation="relu")(classifier_input)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation="relu")(x)
x = layers.Flatten()(x)
features = layers.Dense(64, activation="relu", name="features")(x)
output = layers.Dense(10, activation="softmax")(features)
classifier = Model(classifier_input, output)
classifier.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("Training Fashion-MNIST Classifier...")
classifier.fit(
    x_cls, y_cls,
    epochs=3,
    batch_size=128,
    verbose=1
)
feature_extractor = Model(
    classifier.input,
    classifier.get_layer("features").output
)

# Generate 1000 VAE samples
```

```
z_vae = np.random.normal(
    size=(1000, latent_dim)
).astype("float32")
vae_samples = decoder.predict(z_vae, verbose=0)
# Generate 1000 GAN samples
z_gan = np.random.normal(
    size=(1000, gan_latent_dim)
).astype("float32")
gan_samples = generator.predict(z_gan, verbose=0)
real_samples = x_real[:1000]
print("Real Images:", real_samples.shape)
print("VAE Images:", vae_samples.shape)
print("GAN Images:", gan_samples.shape)
# Classifier-based Inception Score
def inception_score(images):
    predictions = classifier.predict(
        images, batch_size=128, verbose=0
    )
    predictions = np.clip(predictions, 1e-10, 1)
    marginal = predictions.mean(axis=0)
    kl = np.sum(
        predictions *
        (np.log(predictions) - np.log(marginal)),
        axis=1
    )
```

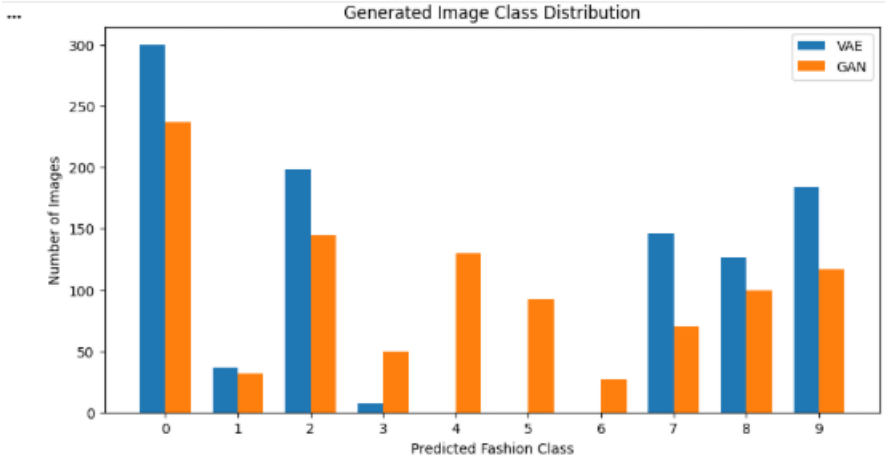
```
    return np.exp(np.mean(kl))
# FID calculation
def calculate_fid(real, generated):
    real_features = feature_extractor.predict(
        real, batch_size=128, verbose=0
    )
    generated_features = feature_extractor.predict(
        generated, batch_size=128, verbose=0
    )
    mu1 = real_features.mean(axis=0)
    mu2 = generated_features.mean(axis=0)
    sigma1 = np.cov(real_features, rowvar=False)
    sigma2 = np.cov(generated_features, rowvar=False)
    diff = mu1 - mu2
    covmean = sqrtm(sigma1 @ sigma2)
    if np.iscomplexobj(covmean):
        covmean = covmean.real
    fid = (
        diff @ diff +
        np.trace(sigma1 + sigma2 - 2 * covmean)
    )
    return float(np.real(fid))
print("\nCalculating Inception Scores...")
vae_is = inception_score(vae_samples)
gan_is = inception_score(gan_samples)
```

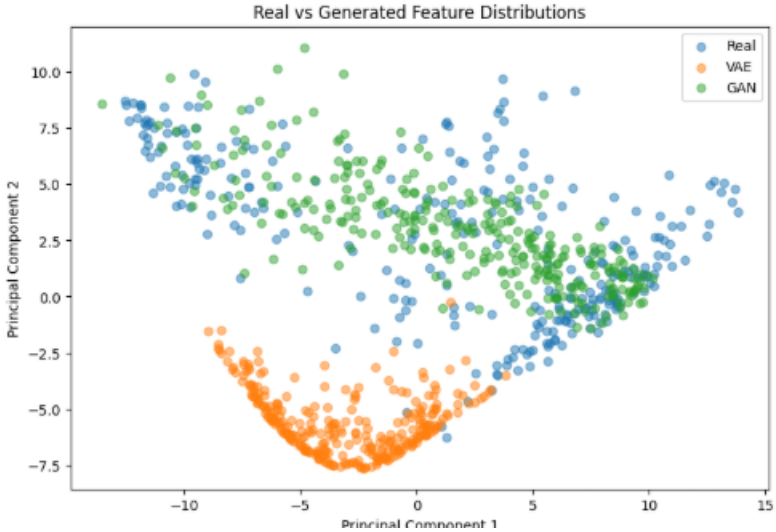
```
print("VAE Inception Score:", vae_is)
print("GAN Inception Score:", gan_is)
print("\nCalculating FID...")
vae_fid = calculate_fid(real_samples, vae_samples)
gan_fid = calculate_fid(real_samples, gan_samples)
print("VAE FID:", vae_fid)
print("GAN FID:", gan_fid)
# Diversity analysis
def diversity_analysis(images):
    predictions = classifier.predict(
        images, batch_size=128, verbose=0
    )
    labels = np.argmax(predictions, axis=1)
    counts = np.bincount(labels, minlength=10)
    probabilities = counts / counts.sum()
    nonzero = probabilities[probabilities > 0]
    entropy = -np.sum(nonzero * np.log(nonzero))
    return counts, entropy
vae_counts, vae_diversity = diversity_analysis(vae_samples)
gan_counts, gan_diversity = diversity_analysis(gan_samples)
print("\nVAE Class Distribution:", vae_counts)
print("GAN Class Distribution:", gan_counts)
print("VAE Diversity Entropy:", vae_diversity)
print("GAN Diversity Entropy:", gan_diversity)
# Class distribution graph
```

```
classes = np.arange(10)
width = 0.35
plt.figure(figsize=(10, 5))
plt.bar(classes - width/2, vae_counts, width, label="VAE")
plt.bar(classes + width/2, gan_counts, width, label="GAN")
plt.xlabel("Predicted Fashion Class")
plt.ylabel("Number of Images")
plt.title("Generated Image Class Distribution")
plt.xticks(classes)
plt.legend()
plt.show()
# PCA visualization
real_features = feature_extractor.predict(
    real_samples[:300], verbose=0
)
vae_features = feature_extractor.predict(
    vae_samples[:300], verbose=0
)
gan_features = feature_extractor.predict(
    gan_samples[:300], verbose=0
)
all_features = np.vstack([
    real_features,
    vae_features,
    gan_features
```

```
])
pca = PCA(n_components=2)
reduced = pca.fit_transform(all_features)
plt.figure(figsize=(9, 6))
plt.scatter(
    reduced[:300, 0],
    reduced[:300, 1],
    label="Real",
    alpha=0.5
)
plt.scatter(
    reduced[300:600, 0],
    reduced[300:600, 1],
    label="VAE",
    alpha=0.5
)
plt.scatter(
    reduced[600:, 0],
    reduced[600:, 1],
    label="GAN",
    alpha=0.5
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Real vs Generated Feature Distributions")
```

```
plt.legend()
plt.show()
# Final comparison table
results = pd.DataFrame({
    "Model": ["VAE", "GAN"],
    "Inception Score": [vae_is, gan_is],
    "FID": [vae_fid, gan_fid],
    "Diversity Entropy": [
        vae_diversity,
        gan_diversity
    ]
})
print("\nFINAL MODEL COMPARISON")
print(results.round(4))
# Recommendation based on measured FID
if vae_fid < gan_fid:
    print("\nRecommendation: VAE has the lower FID.")
elif gan_fid < vae_fid:
    print("\nRecommendation: GAN has the lower FID.")
else:
    print("\nBoth models have equal FID.")
print("Also consider Inception Score, diversity and image quality.")
print("\nExperiment 30 Completed Successfully")
```





RESULT

Thus, VAE and GAN models were evaluated using 1,000 generated images from each model. Classifier-based Inception Score, feature-based FID, class diversity, and PCA visualisation were used to compare their performance.

Implementation link:

https://colab.research.google.com/drive/17N-qB3Bxcxz0dvYVXb8m528btdfQXLBs#scrollTo=Le2eIFbuJjwr